

SDN - NETWORK UTILIZATION MONITOR

REPORT WITH SCREENSHOTS

NAME: Achintha MB

SRN: PES1UG24CS019

SECTION: A

PROBLEM STATEMENT:

To implement an SDN-based network monitoring solution using the Ryu controller and Mininet that demonstrates real-time traffic visibility through OpenFlow 1.3 flow rule design and packet-in event handling.

PROJECT OVERVIEW:

- Objective: The project implements an SDN-based network monitoring solution using the Ryu controller and Mininet to achieve real-time traffic visibility.
- Logic Implementation: The controller manages the network by handling Packet-In events for host connectivity and periodically "pulling" traffic data from switches using OpenFlow 1.3 PortStatsRequests.
- Topology Design: A custom topology was built with a Remote Controller, one Open vSwitch, and two hosts, ensuring a clear separation between the Control Plane and Data Plane.
- Functional Correctness: Successful communication was validated through pingall (reachability) and throughput was analyzed in real-time by calculating Kbps from byte-count differences.
- Performance Tools: Network behavior and the OpenFlow handshake were verified using Wireshark for packet inspection and iperf for stress-testing bandwidth.
- SDN Benefits: This project demonstrates the core SDN advantage of programmability, allowing a centralized software "brain" to monitor and report link utilization without manual switch configuration.

EXECUTION STEPS:

- Environment Setup: Install dependencies on your Ubuntu VM:
`sudo apt update && sudo apt install mininet ryu python3-pip -y`
- Launch Controller: Start the monitoring application in the first terminal:
`ryu-manager monitor_controller.py`
- Start Topology: In a second terminal, launch the custom network:
`sudo python3 topology.py`
- Verify Connectivity: In the Mininet CLI, run a connectivity check:
`mininet> pingall`
- Generate Traffic: Run a bandwidth test to trigger utilization spikes in the controller terminal:
`mininet> h1 iperf -s &`
`mininet> h2 iperf -c 10.0.0.1 -t 10`
- Cleanup: If the network fails to restart, clear the environment:
`sudo mn -c`

SCREENSHOTS:

1. Real-time Port Utilization Monitoring

```
Port 1: 1.84 Kbps
Port 2: 1.37 Kbps
Port 4294967294: 0.00 Kbps

--- DPID 1 Utilization Report ---
Port 1: 0.23 Kbps
Port 2: 0.34 Kbps
Port 4294967294: 0.00 Kbps

--- DPID 1 Utilization Report ---
Port 1: 85411.70 Kbps
Port 2: 1681.85 Kbps
Port 4294967294: 0.00 Kbps

--- DPID 1 Utilization Report ---
Port 1: 103093.94 Kbps
Port 2: 1965.46 Kbps
Port 4294967294: 0.00 Kbps

--- DPID 1 Utilization Report ---
Port 1: 47471.90 Kbps
Port 2: 1023.52 Kbps
Port 4294967294: 0.00 Kbps

--- DPID 1 Utilization Report ---
Port 1: 0.11 Kbps
Port 2: 0.28 Kbps
Port 4294967294: 0.00 Kbps

--- DPID 1 Utilization Report ---
Port 1: 0.28 Kbps
Port 2: 0.00 Kbps
Port 4294967294: 0.00 Kbps
```

2. Pingall and iperf results

```
[04/23/26]seed@VM:~$ export PATH=$PATH:/home/seed/.local/bin
[04/23/26]seed@VM:~$ sudo python3 topology.py
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet> h1 iperf -s &
mininet> h2 iperf -c 10.0.0.1 -t 10
-----
Client connecting to 10.0.0.1, TCP port 5001
TCP window size: 629 KByte (default)
-----
[ 3] local 10.0.0.2 port 38212 connected with 10.0.0.1 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 3]  0.0-10.4 sec   138 MBytes    111 Mbits/sec
mininet>
```

3. Flow table:

```
[04/23/26]seed@VM:~$ export PATH=$PATH:/home/seed/.local/bin
[04/23/26]seed@VM:~$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1
cookie=0x0, duration=481.109s, table=0, n_packets=56279, n_bytes=148156074, priority=0 actions=CONTROLLER:65535
[04/23/26]seed@VM:~$
```

4. Wireshark:

